

多智能体协作规划系统

项目讲解手册（从零理解 Agent 怎么搭）

适用：课程大作业 / 比赛说明 / 团队（A · B · C）传阅

整理日期：2026-07-13

一、这个项目到底在做什么（先建立全局直觉）

它是一个“小组作业规划器”：

- 输入：一门课的名字 + 作业要求 + 组员名单（每人带技能标签）+ 提交截止日。
- 输出：一份完整协作方案——

任务怎么拆、谁讲谁答、什么时候该干完、关键路径在哪、模拟答辩会问什么。

它叫“多智能体（Multi-Agent）”，但别被词吓到。在本项目的代码里，一个 Agent 不是神秘实体，它就是一个 Python 类，干四件事：

- 1) 持有一段固定的 system_prompt（给 LLM 的“岗位说明书 + 输出规矩”）；
- 2) 把输入拼成一段 user_prompt；
- 3) 调用 LLM，并强迫它只能返回 JSON，且 JSON 字段长得跟某个 Pydantic 模型一模一样；
- 4) 把这个 JSON 变成 Python 对象返回。

“多智能体”= 好几个这样各管一摊的类，由一个 Coordinator（总调度）按顺序串起来。所谓“智能”90% 在 LLM 和 Prompt 里，Agent 类本身只是管道和契约。

二、一个 Agent 系统“从零搭建”的真实顺序

本项目文档 docs/任务拆解.docx 中的“70 个小任务”将整个项目切成 6 大块：

- 产品设计（定位、PRD、Agent 设计说明书、流程图）
- 软件工程（目录、Git、配置、依赖）
- 数据层（数据模型、Schema、数据流）
- AI 能力（大模型接入、Prompt 体系、Structured Output）
- Multi-Agent 核心：Coordinator / Planner / Matcher / Timeline / QA / Report / Interview / 后期 Knowledge、Reflection、Memory
- 知识库（素材整理、RAG，比赛阶段才做）

这 70 个任务又按“能不能演示”分三档：

- A 类（必做，跑不通就没法演示）：A1 LLM 封装、A2 主链路、A3 QA 矩阵、A4 倒排时间线+关键路径、A5 只读 Web。
- B 类（加做，拉开差距）：B1 答辩模拟、B2 Memory、B3 完整角色匹配、B4 协作图动态修改。
- C 类（比赛再加）：RAG、平台接入、反思 Agent、部署。

对应的 5 天节奏（D1→D5）：

- D1：定数据模型 + JSON 契约（第 0 步）+ LLM 封装 + FastAPI 骨架 + 最小垂直切片跑通。
- D2：Planner + Matcher 的 Prompt 迭代，主链路 A2 跑通。
- D3：Timeline + QA 矩阵 + 可解释性。
- D4：只读 Web + B1 答辩模拟 + 边界用例测试。
- D5：联调 + 演示彩排。

最关键的一步是第 0 步：先定死 JSON 契约（schemas.py），再让 A、C 各自开写。契约不定，多人后续一集成必然对不上——

这就是“人做架构”最值钱的一步。团队前期已用自动生成工具搭出 D1 – D4

的代码骨架，因此现在看到的是“壳子有了，但灵魂（契约细节、Prompt 质量、校验逻辑）还空着”。

三、核心概念逐个讲透

3.1 「数据结构」到底是什么

在这里，“数据结构”就是 `app/models/schemas.py` 里的那些 Pydantic 模型（BaseModel 的子类）。它们定义了系统里每个数据的“形状”。因为整个系统只传 JSON，所以“数据结构”和“JSON 接口”是同一个文件的两面。

入口数据（用户给系统的）：

- CourseInfo：课程名 + 描述。{name, description}
- TeamMember：组员姓名 + 技能标签列表。{name, skill_tags:[...]}
- AssignmentInput：整套系统的入口。含 course / members / deadline / additional_requirements。

Planner 输出（任务拆解）：

- SubTask：{id, name, description, estimated_hours, dependencies:[...], required_skills:[...]}
- dependencies 是“依赖的其他任务 id 列表”，让任务之间构成一张有向无环图（DAG）。后面 Timeline 算关键路径、B4 做“环检测”都靠它。
- PlanOutput：{tasks:[SubTask], summary, reasoning}

Matcher 输出（QA 责任矩阵）：

- QAAssignment：单个任务的答辩分工。{task_id, task_name, chapter, presenter, qa_primary, qa_support:[...], reasoning}
- QAOutput：{assignments:[QAAssignment], note}

Timeline 输出（时间线）：

- TimelineTask：{task_id, name, start_date, end_date, is_critical:bool, assigned_to:[...]}
- TimelineOutput：{tasks:[...], critical_path:[...], total_days, note, reasoning}, critical_path 即“哪几个任务延期会让整个项目延期”。

出口与错误：

- ReportOutput：{summary, timeline_section, qa_matrix_section, risk_note}
- FullPlan：系统最终输出，把上面所有东西 + 原始输入包在一起。{input, plan, timeline, qa_matrix, report, version}
- AgentError：统一错误格式。{agent, error_type, message, recoverable:bool}，每个 Agent 失败时返回它而不是抛异常。

一句话：`schemas.py` 是全系统的“数据字典 + 接口契约”。改一个字段名，全项目受影响——所以必须最先定、且大家共用。

3.2 「JSON 接口」到底是什么（契约 + 机制）

第一层：契约（长什么样）—— Planner 必须吐 PlanOutput 这种 JSON，Matcher 必须吃 PlanOutput、吐 QAOutput，依此类推。这就是 Agent 之间的“接口”。Coordinator 拿到每个 Agent 的返回值后用 Pydantic 做 validate（类型、必填字段检查），不对就重试或降级。

第二层：机制（怎么逼 LLM 真的吐这种 JSON）—— 这是 `app/llm/client.py` 的 `chat_structured` 方法核心：

```
resp = self._client.beta.chat.completions.parse(
    model=self.model,
    messages=[{"role":"system","content":system_prompt},
              {"role":"user","content":user_prompt}],
    response_format=response_model, # 直接把 Pydantic 类传进去
    temperature=0.3,
)
```

```
raw = resp.choices[0].message.content
return response_model.model_validate_json(raw) # 字符串 -> 带类型检查的 Python 对象
```

普通 `chat.completions` 返回自由文本，没法保证是合法 JSON。而 `beta.chat.completions.parse +`

`response_format=Pydantic` 类是 OpenAI 的 Structured Output（结构化输出）：模型被约束成只能生成符合该 schema

的 JSON，再 `model_validate_json` 变成对象。

失败处理也在这一层：整个调用包在 `for attempt in range(max_retries)` 里（默认重试 3 次）。一旦炸了（空回复、格式错、超时）就记日志重试；全失败才返回 `AgentError(recoverable=True)`。这就是为什么系统再怎么抽风也不会直接崩——它在基础设施层就把“LLM 不可靠”接住了。

- `interview_sim.py` 的 `chat_text` 是另一个方法：不强制 JSON（答辩问题本来就是一段话），返回 `str` 或 `AgentError`。

3.3 什么叫做「LLM 封装」

LLM = 大语言模型（GPT 那类）；封装 =

把乱七八糟的内部实现藏起来，只对外留一个干净好用的接口。所以“LLM 封装”=

把“怎么跟模型说话”这件麻烦事，包成一个简单好用的类，让业务代码不用关心底层细节。

“不封装”会怎样：每个 Agent 都直接调 OpenAI SDK，Key

每个文件抄一遍、模型名写死、失败怎么重试不知道、返回怎么解析成对象各写一遍——多个 Agent 重复多遍脏活，换模型得改多处。

封装之后（本项目的 app/llm/client.py）：Agent 只写

```
llm = LLMClient()
result = llm.chat_structured(
    system_prompt=SYS,
    user_prompt=USR,
    response_model=PlanOutput,
)
```

Key 从哪读、模型是谁、失败重试几次、返回怎么解析成 PlanOutput、出错返回什么——全在 client.py 内部消化，Agent 完全不知道也没必要知道。类比：把发动机套上一个只有“油门/刹车”的壳，司机（Agent）只踩踏板，发动机坏了换型号只动壳子内部。

所以本项目中“LLM 封装”具体指的就是 app/llm/client.py 这个文件，对外提供两个方法：chat_structured（要 JSON 结构时用）和 chat_text（只要一段话时用，如答辩模拟）。

3.4 「多智能体」到底是什么（Agent 类 = 管道 + 契约）

每个 Agent 是一个文件（继承 BaseAgent），只关心“我吃什么 JSON、吐什么 JSON、用哪个

Prompt”。它们之间不直接互相调用，全由 Coordinator 串。BaseAgent 提供公共调用逻辑 _call_llm：把 user_prompt 丢给 LLMClient.chat_structured，自动拿到结构化对象。子类只要设 system_prompt、response_model

两个类变量，再实现自己的 run() 即可。

所以“多智能体”的本质 = 多个“各管一摊、只通过 JSON

契约对接”的管道，加一个总调度把它们按顺序接起来。难点不在“多”，而在“契约对齐 + 失败降级 + 顺序编排”。

3.5 什么叫做「LangChain 架构」

澄清误会：LangChain 不是模型，它是一个“搭 LLM 应用的框架”，把常见零件预先做好让你拼更快。核心积木：

- PromptTemplate：把变量填进提示词模板 → 本项目中就是 app/llm/prompts.py。
- LLM / ChatModel：对模型的封装 → 本项目中就是 app/llm/client.py 的 LLMClient。
- OutputParser：把模型返回字符串变成结构化对象 → 本项目中就是 schemas.py + model_validate_json。
- Chain：把“填模板 → 调模型 → 解析输出”串成一步 → 本项目中就是 BaseAgent._call_llm + 各 Agent 的 run()。
- Agent + Tools：让模型自己决定调哪个工具（查库、算数），这是 LangChain 最有名的能力，也是 Agent 真正“自主”的地方 → 本项目规划里的 C5 Tool Calling，目前没做。
- Memory：跨多轮对话记住之前说过的话 → 本项目规划里的 B2 / C 类 Memory，目前没做。
- Retriever / VectorStore：从知识库找资料喂给模型（RAG） → 本项目规划里的 C1 / C3 知识库，比赛阶段才做。

本项目与 LangChain 对照表：

LangChain 概念	本项目里的对应	说明
ChatModel（模型封装）	app/llm/client.py 的 LLMClient	一模一样的作用
PromptTemplate	app/llm/prompts.py	模板 + .format() 填值
OutputParser	schemas.py + model_validate_json	结构化输出
Chain（单步链）	BaseAgent + 各 Agent 的 run()	一步流水线
Agent/Workflow（多步编排）	Coordinator + 多个 Agent	总调度串主链路
Memory	memory/ 目录 + 规划中的 B2	尚未实现

Retriever / RAG	规划中的 C1 / C3	比赛阶段
-----------------	--------------	------

关键澄清：本项目并未引入 LangChain（requirements.txt 中无该依赖）。本项目使用原生 openai + pydantic 手搓了一套 “LangChain

风格”的轻量框架。好处：依赖少、完全可控、便于排查问题，且每一层的实现都清晰可读——这正是学习 agent 最值钱的部分。代价：重试、解析、编排等底层逻辑需自行实现（LangChain 已替你写好），但本作业这个规模的 MVP 完全够用，且答辩时可清楚说明“每一层均为自主设计”。

一句话：LangChain 是现成的封装方案，本项目则以相同思路自行实现了一遍。因此团队不是“要不要学 LangChain”的问题，而是已经用最硬核的方式把 LangChain 干的事做了一遍。

3.6 Tool / Function Calling（工具调用）—— Agent 和“普通调一次模型”的本质区别

普通调模型：你给一段话，模型回一段话，结束。模型“只会说话”。

工具调用：你事先告诉模型“你有三个函数可用：查天气(query)、算日期差(a,b)、查数据库(sql)”。模型在回答时，如果觉得需要，就会返回一个“我要调用 查天气(北京)”的结构化指令；你的程序真的去执行这个函数、拿到结果，再连同结果喂回模型，模型据此给出最终答案。

这一步为什么关键：它让模型从“只能聊”变成“能动手”。这才是 Agent 真正“自主”的来源——模型自己决定下一步调哪个工具。本项目规划中的 C5（Tool Calling）就是这一步；当前主链路里 Planner/Matcher 都是“纯文本进、JSON 出”，还没接工具。典型工具例子：

- 查日历/算关键路径（其实 Timeline 的关键路径现在让 LLM 顺手给，理想是用代码工具算，更稳）
- 查往届作业经验库（RAG 检索，见 3.7）
- 写入/读取 Memory（存计划、读历史，见 3.10）

3.7 RAG（检索增强生成）—— 本项目规划中要接入的方向

问题：LLM 仅具备训练时学到的通用知识，对本课程、本小组的往届经验并不了解，硬问会“幻觉”（胡编）。

RAG 的做法：先建一个“知识库”（把往届作业要求、优秀方案、常见坑整理成文档），用户提问时，先去知识库里“搜出最相关的几段”，再把“问题 + 搜到的资料”一起喂给模型，让它基于资料作答。这样回答更准、可溯源、不胡编。

在本项目中，规划里的 C1 / C3 知识库即为 RAG 的雏形。后续计划大概会新增：

- knowledge/ 目录：存素材（往届经验、评分标准）。
- embedding + 向量库：把资料变成向量，按语义相似检索（见 3.8）。
- 一个 Retrieval Agent：主链路前先检索，把相关资料塞进 Planner/Matcher 的 prompt。

注意：RAG 不是替代主链路，而是给主链路的某个 Agent “外挂”一个知识源，让它吐的 JSON 更有依据。

3.8 Embedding（向量化）—— RAG 检索的前置步骤

文字本身计算机不好直接比“像不像”。Embedding 是用一个模型把一段文字变成一串数字（向量），语义相近的文字，向量在空间里也离得近。于是“检索”就变成“算向量距离，找最近的几段”。

举例：用户问“任务怎么拆更合理”，知识库里“优秀方案的任务拆解范例”那段，和它的向量距离很近，就被检索出来。没有

Embedding，只能做关键词匹配（问“拆”就必须文档里恰好出现“拆”字），效果差很多。

3.9 Chain（链）—— 把多步串成流水线

Chain = 把“模板填值 → 调模型 → 解析输出”固定成一步可复用的单元。本项目中 BaseAgent._call_llm 与各 Agent 的 run() 即为一条单步链；Coordinator

把多条链（Planner → Matcher → Timeline → Reporter）再串成一条“总链”。Chain 的思想就是“可组合”：小链拼大链，改一处不影响别处。

3.10 Memory（记忆）—— 跨轮不“失忆”

默认 LLM 每调一次都是“金鱼脑”，不记得上一次说什么。Memory 就是让系统把历史存下来（文件 / 数据库），下次带着历史一起问。本项目规划中的 B2 Memory（保存/加载计划 JSON）就是这个：把生成的 FullPlan 存到 memory/ 目录，下次能接着改，而不是每次从零来。

3.11 Token（令牌）—— 计费与长度的基本单位

模型不是按“字”算，而是按 token 算。一句话会被切成若干 token（中文大概 1-2 字一个 token，英文一个词几个 token）。prompt 越长、输出越长，token 越多，费用越高、也更容易触到上下文上限。写 prompt 时要“既把规矩说清，又别啰嗦”，本质就是在控 token。

3.12 Fine-tuning（微调）—— 本作业现阶段用不到

微调是用专属数据重新训练（或轻度训练）模型，让它更适配你的领域。代价高、需要数据、需要训练流程。本作业依靠“选好模型 + 写好 Prompt + 接入 RAG”即可满足需求，现阶段无需涉及微调，不必被这个词吓到。

四、目录为什么这么搭（按“关注点分离”分层）

一句话：按关注点分层，让多人能并行、且互不踩。目录结构：

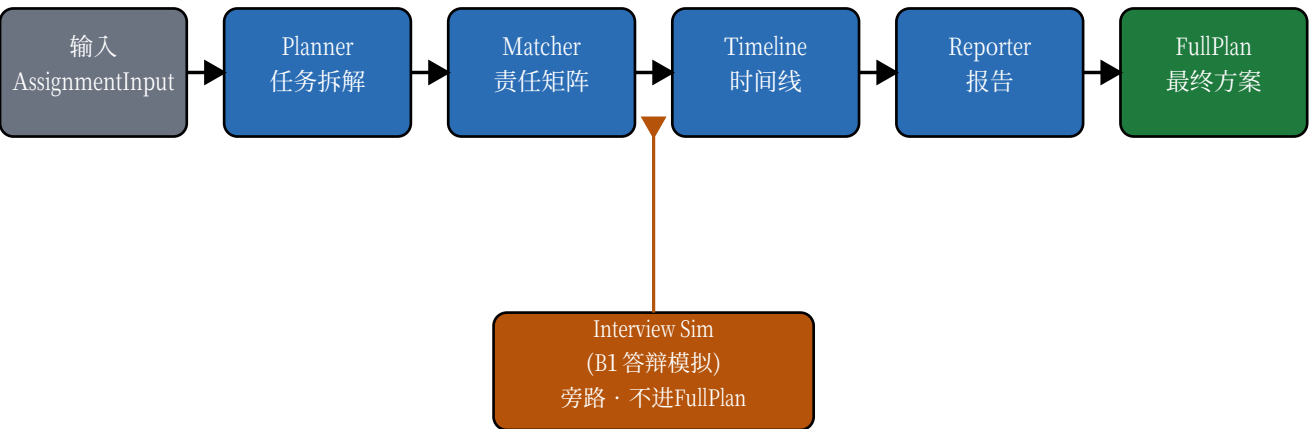
```
app/
├── config.py      # 配置层：Key/模型/端口，从 .env 读
├── models/schemas.py # 数据层：所有 JSON 契约（第 0 步）
├── llm/           # 基础设施层：LLM 封装 + Prompt 模板
│   ├── client.py  # A1：调用+重试
│   ├── prompts.py # 所有 Agent 的 Prompt
├── agents/        # 智能层：每个 Agent 一个文件
│   ├── base.py    # 基类（公共调用逻辑）
│   ├── planner.py # A 负责
│   ├── matcher.py # C 负责
│   ├── timeline.py # C 负责
│   ├── reporter.py
│   └── interview_sim.py # B1, B 角负责
├── coordinator.py # 编排层：唯一知道全链路的地方
├── main.py        # 接入层：FastAPI 入口
├── web/           # 接入层：路由 + 前端
│   ├── routes.py
│   ├── templates/index.html
│   └── static/style.css
```

为什么要这么分，逐层说清：

- models/ 单独成层：数据形状只有一处定义。改字段改一处，全项目跟着变；A、C 不用各自定义一份，避免“你说你的字段、我读我的字段”。这是并行不出乱子的根基。
 - llm/ 单独成层：所有“怎么调模型”的细节（重试、base_url、Structured Output）只在此处。换模型/换供应商/加 Token 限制只改 client.py，Agent 一行不动。Prompt 集中放 prompts.py，方便 A、C 各自迭代自己的那段。
 - agents/ 每个 Agent 一个文件：A 改 planner.py、C 改 matcher.py/timeline.py，互不 import 对方业务代码，只通过 schemas.py 的 JSON 对接。这直接对应分支策略——各人在自己分支改自己文件，合并几乎不冲突。
 - coordinator.py 单独成层：唯一“知道整条流程”的地方（先 Planner、再 Matcher、再 Timeline、再 Reporter）。流程怎么串、某步失败怎么降级全集中可改；Agent 各自傻瓜化，只管“给我输入、还你输出”。
 - web/ / main.py 是接入层：只是“把 HTTP 请求转成 AssignmentInput、把 FullPlan 转成 HTTP 响应”。就算没有 Web，系统照样能跑——tests/ 里就是直接 Coordinator().run(...) 调用的。这就是“纵切深井”：先让核心链路在命令行/test 跑通，Web 只是最后套的壳。
- 依赖方向：上层调下层，循环依赖为 0 —— 这是工程上讲“目录搭得对”的硬指标。

五、三张架构图（对应最初生成的图，重新绘制）

图1 · 多智能体主链路



箭头=数据流方向；橙色虚线=旁路功能（不进入 FullPlan）

图2 · 目录分层与职责（箭头=依赖方向，上层→下层）

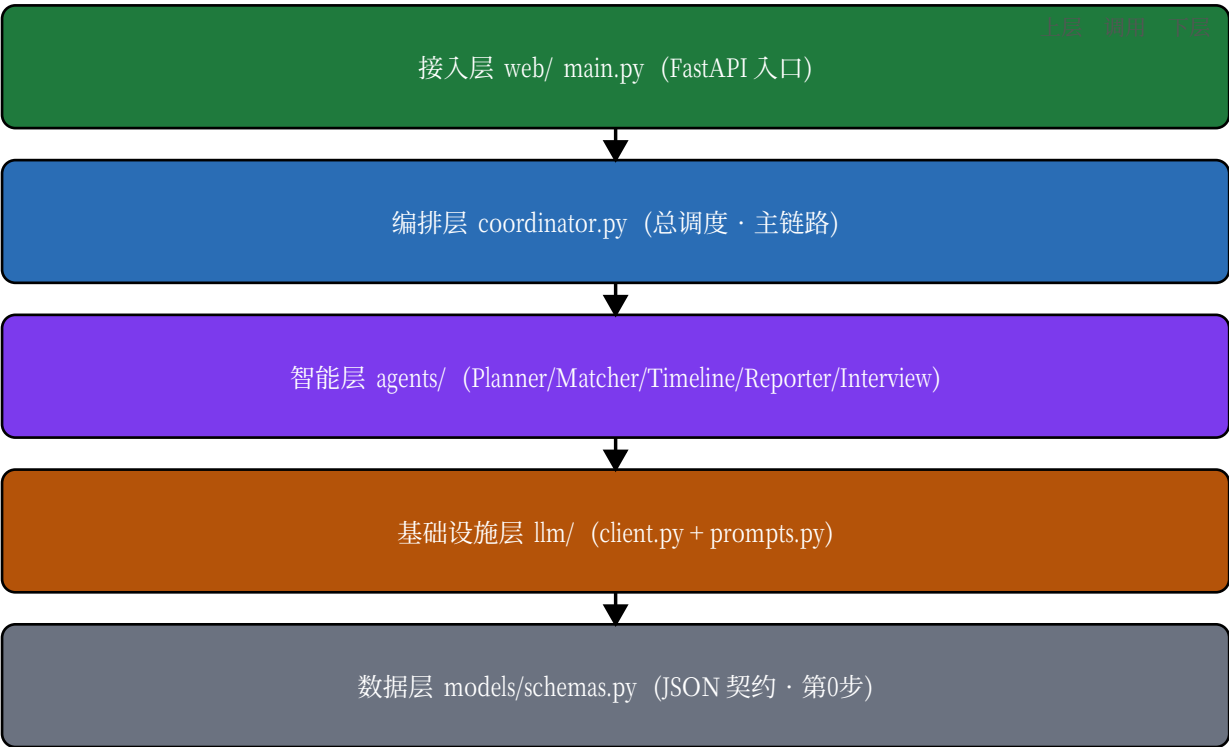
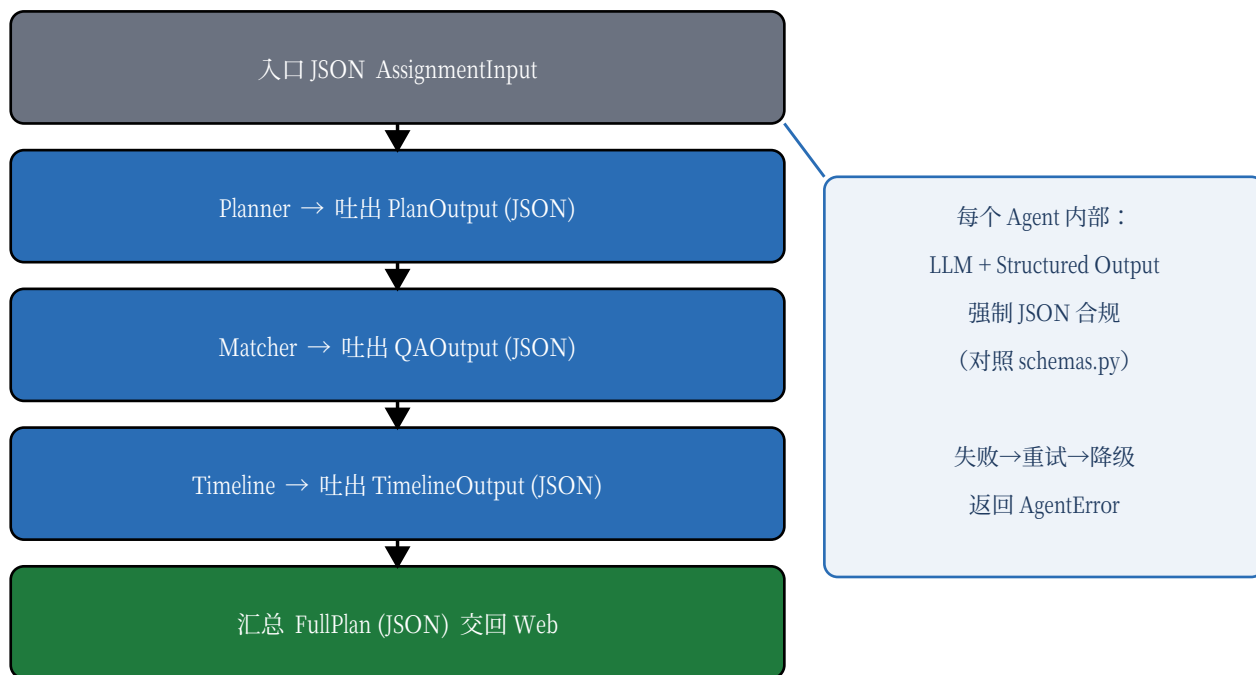


图3 · JSON 接口契约（信封式：每个 Agent 吃 JSON、吐 JSON，最终汇总）



六、每个文件逐个讲：做什么、怎么做、谁负责

按“运行时调用顺序”走一遍：

- 1) app/config.py (B角负责)：从 .env 读 LLM_API_KEY / LLM_BASE_URL (默认官方) / LLM_MODEL (默认 gpt-4o-mini) / LLM_MAX_RETRIES=3 / APP_HOST/PORT。BASE_DIR 用 Path(__file__).resolve().parent.parent 自动定位项目根，路径不写死。本地要 cp .env.example .env 再填 Key。
- 2) app/models/schemas.py (B角负责，第0步)：上一节全部模型，是全项目“数据字典+接口契约”。date 类型字段在 JSON 里是字符串，Pydantic 自动 parse；FullPlan 把四个 Agent 输出聚合，是 Web 最终返回类型。
- 3) app/llm/client.py (B角负责，A1)：LLMClient 每个 Agent 可独立实例化。chat_structured 负责“调模型→拿 JSON→校验成对象→失败重试”；chat_text 负责自由文本。它不关心业务，只关心可靠拿到结构化结果。
- 4) app/llm/prompts.py (A 写 Planner 段、C 写 Matcher/Timeline 段、B角写 Interview 段)：集中所有 Prompt。*_SYSTEM 是岗位说明书，*_USER_TEMPLATE 是带占位符的模板(如 {course_name})，运行时 .format(...) 填值。这是最需由对应负责人反复迭代之处，文档指出“Prompt 迭代比预期慢 2-3 倍”且“须由人亲自完成”。
- 5) app/agents/base.py (B角负责，骨架)：BaseAgent(Generic[T]) 基类，只干一件事——把 system_prompt + user_prompt + response_model 交给 LLMClient 拿结构化结果 (_call_llm)。子类设两个类变量 + 写 run() 即可。
- 6) app/agents/planner.py (A 端到端负责)：PlannerAgent(BaseAgent[PlanOutput])。run() 把课程/组员/截止日填进 PLANNER_USER_TEMPLATE，调 LLM 返回 PlanOutput (5-8 个子任务)。TODO：输出校验 (task id 唯一性、依赖不指向不存在的任务)——这是 A 的职责，集成前必须补齐。
- 7) app/agents/matcher.py (C 端到端负责)：MatcherAgent(BaseAgent[QAOutput])。把 PlanOutput + 组员信息喂 LLM，返回每个任务的答辩分工 (主讲/主答/辅答 + 一句为什么)。TODO：输出校验 + B3 完整角色匹配 (表达/应变标签、轮值制)。
- 8) app/agents/timeline.py (C 端到端负责)：TimelineAgent(BaseAgent[TimelineOutput])。依任务和截止日让 LLM 倒排起止日期 + 关键路径。TODO：关键路径计算逻辑 (现靠 LLM 顺手给，不稳；理想用代码算法算)。
- 9) app/agents/reporter.py (B角负责)：ReporterAgent(BaseAgent[ReportOutput])。把 Plan+Timeline+QA 三份 JSON 拼一起，让 LLM 生成人读报告文本，是链路末端。
- 10) app/agents/interview_sim.py (B角负责，B1)：InterviewSimAgent 用 chat_text 让 LLM 扮评委，基于 Plan+QA 生成 10-15 道答辩题。它不进主链路的 FullPlan，是旁路。response_model=None，返回 str，出错返回 "[Error] ..."。
- 11) app/coordinator.py (B角负责，编排核心)：持有四个 Agent 实例，run(inp)->FullPlan 按顺序跑：Planner 失败直接 raise (命脉)；Matcher/Timeline/Reporter 失败不崩，降级成空对象 + 写日志。这就是“脆弱步骤可降级、关键步骤必须成功”的容错设计。
- 12) app/main.py + app/web/routes.py (B角负责，A5 只读 Web)：main.py 建 FastAPI，把 routes 挂到 /api，把 web/templates 作静态首页。routes 提供 POST /api/run (收 RunRequest→转 AssignmentInput→调 Coordinator→返 FullPlan) 和 GET /api/health。Web 只是壳，前端现在只能“提交→看结果”，编辑 UI 是 B4 才做。
- 13) tests/ (C 负责测试框架)：test_coordinator.py 用 monkeypatch 把四个 Agent 的 run 换成假数据，验证主链路能拼出 FullPlan (不真调 LLM，快且稳)；test_api.py 用 httpx + ASGITransport 在内存跑 FastAPI，验证 /api/health 返回 200。这是“每日跑通验证”的自动化手段。

七、分支策略（BRANCHES.md）对应谁、怎么并行

- main：稳定版，仅由 B 角（提交人）合并。
- coordinator（B 角）：骨架、LLM 封装、FastAPI、Coordinator、Web、集成、B1、B2。
- agent/planner（A）：只动 planner.py + 自己的 Prompt。
- agent/timeline-qa（C）：只动 matcher.py / timeline.py + 自己的 Prompt。
- fix/*、docs/*：谁用谁建。

并行逻辑：A 只改 planner.py、C 只改 matcher.py/timeline.py、B 角只改 coordinator.py/web/，唯一共同依赖是早已定死的 schemas.py。所以三人各写各分支、互不冲突，最后由 B 角每天 merge 一次到 main 做集成。这也说明为什么“第 0 步定契约”这么重要——契约不定，并行就变成互相覆盖。

八、现状盘点：脚手架做了什么、还差什么

已经做好的（壳子 + 主链路能跑）：

- schemas.py 全模型已定义（第 0 步完成）。
- llm/client.py 封装 + 重试完成（A1 完成）。
- 五个 Agent 类 + coordinator.py 全部写完，主链路逻辑通。
- main.py + web/routes.py 只读 Web 完成（A5 完成）。
- tests/ 两个测试在（mock 版）。

仍为空 / 未做的部分（需由对应负责人亲自动手）：

- planner.py / matcher.py / timeline.py 里三处 # TODO 输出校验没写——集成前必补，否则下游可能拿到非法数据。
- prompts.py 为初版，其质量需由负责人反复迭代（文档说这是真正瓶颈）。
- B2 Memory（保存/加载计划 JSON）、B4 协作图动态修改（add_task/remove_task/reassign/change_ddl + 环检测）完全没做。
- timeline 关键路径现在靠 LLM 顺手给，没用代码算法算。
- 尚未做过一次真实跑通（需先填写 .env 中的 Key，再经 POST /api/run 或 Coordinator().run() 实测）。

九、一条“研究 agent 如何从零搭建”的动手路线（供搭建者实践）

- 1) 先读 schemas.py 三遍，把它当“数据字典”背下来——这是地基。
- 2) 单独运行 llm/client.py：写一段小程序用 chat_structured 让模型返回一个小 Pydantic 模型，观察 Structured Output 怎么把自由文本逼成 JSON。
- 3) 单独运行 planner.py：输入一门真实课程，看它吐的 PlanOutput 长啥样；然后亲手补齐那个 TODO 校验（id 唯一、依赖合法），这是第一次为 Agent 写入“灵魂”逻辑。
- 4) 在 coordinator.py 里加一行 log，观察主链路四步依次执行的顺序。
- 5) 启动 Web（python -m app.main），用 POST /api/run 走完整链路，把返回 JSON 和 schemas.py 对一遍。
- 6) 最后再迭代 prompts.py，把 Planner 的输出质量磨上去。

十、Git 协作与合成流程（团队怎么并行不打架）

本项目采用“按角色分分支、由提交人统一合并”的工作流（见 BRANCHES.md）。核心原则：每人只在自己分支动自己负责的少量文件，唯一共享且先对齐的契约是 schemas.py；提交人（B 角）每天把各分支合并进 main 做一次集成。

分支规划：

- main：稳定版，只有提交人能合并。
- coordinator（B 角）：骨架、LLM 封装、Coordinator、Web、集成。
- agent/planner（A）：仅动 planner.py 与自己的 Prompt。
- agent/timeline-qa（C）：仅动 matcher.py / timeline.py 与自己的 Prompt。

日常流程（以 A 在 planner 分支工作为例）：

```
# 1) 拉代码并建自己的分支（每人一次）
git clone <repo-url>
cd competition
git checkout -b agent/planner

# 2) 在自己分支上改代码、提交
git add app/agents/planner.py app/llm/prompts.py
git commit -m "feat(planner): 补输出校验 + 迭代 Prompt"

# 3) 推送自己的分支（便于提交人合并）
git push -u origin agent/planner
```

合并流程（由提交人 B 角执行，每天一次）：

```
# 切回 main，先拉最新
git checkout main
git pull origin main

# 合并 A 的分支；无冲突则自动完成
git merge agent/planner

# 若 schemas.py 出现冲突：说明契约没先对齐，需三人线下确认后手工改
# 再合并 C 的分支
git merge agent/timeline-qa

# 合并完跑测试 + 一次真实调用验证
pytest
python -m app.main # 或 POST /api/run
git push origin main
```

两条铁律：

- ① schemas.py（JSON 契约）若要改动，必须三人先对齐再各自提交，否则合并必冲突、集成必炸。
- ② .env（含 API Key）、本地 memory/产物不要提交；建议在 .gitignore 中忽略 .env、__pycache__ / 与 .workbuddy/。

这种“各改各的分支、仅共享契约、每日统一合并”的节奏，恰好呼应了目录分层与第 0 步定契约的设计——分层和契约是前因，分支策略是后果。

十一、如何本地跑通一次真实调用（验证链路真的通）

前面讲的都是“结构”，这一节讲“真跑一次”，把抽象链路落成可观测的结果。建议分两步：先用 mock 测试确认代码逻辑无误，再用真实 Key 跑一次端到端。

步骤 1 · 准备环境

- 安装依赖：pip install -r requirements.txt（建议在独立虚拟环境中）。
- 配置 Key：cp .env.example .env，编辑 .env 填入 LLM_API_KEY / LLM_BASE_URL / LLM_MODEL 等。
- 忽略项：确认 .gitignore 含 .env、__pycache__/, 避免泄露密钥。

步骤 2 · 先跑 mock 测试（不花 Key、不联网）

```
pytest # test_coordinator / test_api 用假数据验证逻辑能串
```

步骤 3 · 真实调用方式一：命令行直接跑 Coordinator

新建一个临时脚本 run_once.py（或 python -c），直接构造输入、调用主链路：

```
from datetime import date
from app.coordinator import Coordinator
from app.models.schemas import AssignmentInput, CourseInfo, TeamMember

inp = AssignmentInput(
    course=CourseInfo(name="软件工程", description="小组项目，含模拟答辩"),
    members=[TeamMember(name="张三", skill_tags=["前端", "PPT"]),
             TeamMember(name="李四", skill_tags=["Python", "后端"])],
    deadline=date(2026, 7, 20),
    additional_requirements="需要模拟答辩",
)
plan = Coordinator().run(inp) # 四步依次执行：Planner→Matcher→Timeline→Reporter
print(plan.model_dump_json(indent=2)) # 对照 schemas.py 看字段是否齐全
```

步骤 4 · 真实调用方式二：启动 Web 走 HTTP

```
# 终端 1：启动服务
python -m app.main

# 终端 2：发请求（注意 deadline 是字符串 "YYYY-MM-DD"）
curl -X POST http://localhost:8000/api/run \
-H "Content-Type: application/json" \
-d '{"course":{"name":"软件工程","description":"小组项目"},
  "members":[{"name":"张三","skill_tags":["前端"]}],
  "deadline":"2026-07-20","additional_requirements":"模拟答辩"}
```

步骤 5 · 看结果、做校验

- 对照 schemas.py：返回 JSON 的字段应与 FullPlan 完全一致（input/plan/timeline/qa_matrix/report/version）。
- 观察降级：若某 Agent 失败，主链路不应整体崩，而应返回降级后的 FullPlan 或 AgentError（这正是 coordinator.py 的容错设计）。
- 若模型吐的 JSON 字段缺失/错乱：回到 prompts.py 迭代对应 Agent 的 Prompt，而非改代码逻辑。

一句话：能跑 mock 不等于能跑真实；每日一次真实 /api/run 或 Coordinator().run()，是验证“链路真的通”的唯一标准。

十二、AI 编程助手（Codex / Claude Code）下载与配置

本项目的代码骨架前期是借助 AI 编程助手（如 Codex、Claude Code）快速搭建的。若团队成员也想在本地使用这类工具来辅助开发，有两种配置思路：方法一是用统一面板（cc switch），方法二是手动直接配置。下面分别说明。

方法一：用 CC Switch 统一管理（推荐，省去手改配置）

CC Switch 是一款跨平台桌面应用（Windows / macOS / Linux），用来统一管理 Claude Code、Codex、Gemini CLI、OpenCode、OpenClaw 等 CLI 工具的“供应商配置”（即把请求发到哪个模型服务、用哪个 Key）。它内置 50+ 供应商预设，选好预设后只需填 API Key，一键切换即可生效，免去手动编辑 settings.json / auth.json / .env 的繁琐。

下载安装（GitHub Releases 取对应包）：

- Windows：下载 .msi 双击安装，或下载绿色便携版 .zip 解压即运行 CC-Switch.exe。
- macOS：下载 .dmg 拖入“应用程序”，或 Homebrew：brew tap farion1231/ccswitch && brew install --cask cc-switch。
- Linux：下载 .deb 用 dpkg -i 安装，或用 .AppImage 直接运行。

配置三步（核心：装 → 加供应商填 Key → 点启用）：

- 1) 启动 CC Switch，它会自动检测本机已装的 CLI 并在系统托盘生成图标。
- 2) 点“+ / Add Provider”添加供应商：选内置预设（填 Key 即可），或手动填 API Key + Base URL。
- 3) 点“启用 / Enable”，程序自动把配置写入对应 CLI 的配置文件（如 Claude Code → ~/.claude/settings.json，Codex → ~/.codex/），无需手改。

优势：Claude Code 支持“热切换”——改完供应商配置立刻生效，不必重启终端；多个 CLI、多个供应商一处管理。

方法二：直接配置（手动安装 CLI + 手动填 Key）

若不想额外装面板，也可以各自手动装 CLI、手动配认证。

A. Codex CLI（OpenAI）

```
# 安装（需 Node.js 18+）
npm install -g @openai/codex
codex --version
```

```
# 登录：首次运行按提示用 ChatGPT 账号登录，或导出 Key
export OPENAI_API_KEY="sk-..." # Linux/macOS
# Windows: setx OPENAI_API_KEY sk-...
codex # 进入交互；/model 可切换模型
```

```
# 审批模式（控制它有多“自主”）
codex --suggest # 默认：只读+提议，改动需你批准
codex --auto-edit # 自动改文件，命令仍需批准
codex --full-auto # 沙箱内全自动（长任务用）
```

注意：Windows 官方支持为实验性，建议在 WSL 中使用。Key 会被写入 ~/.codex/auth.json。

B. Claude Code（Anthropic）

```
# 安装（Node 18+，或原生安装脚本）
npm install -g @anthropic-ai/claude-code
claude --version
```

```
# 登录：运行 claude 后按提示 /login 浏览器授权；或导出 Key
export ANTHROPIC_API_KEY="sk-ant-..." # Linux/macOS
# Windows: setx ANTHROPIC_API_KEY sk-ant-...
```

```
# 云厂商接入（可选）
export CLAUDE_CODE_USE_BEDROCK=1 # 走 Amazon Bedrock
export CLAUDE_CODE_USE_VERTEX=1 # 走 Google Vertex AI
登录后凭据存于 ~/.claude/，一般无需重复登录；切换账号用会话内 /login。
```


两种方法对比：

维度	方法一：CC Switch	方法二：直接配置
配置方式	面板选预设、填 Key、一键启用	手动装 CLI、手改 settings/auth/.env
切换供应商	面板/托盘秒级热切换	改配置后常需重启 CLI
多工具管理	一处管 Claude Code/Codex/Gemini 等	每个工具各自配，分散
上手成本	低（可视化）	中（需懂各 CLI 配置文件）
可控性	受面板约束	完全自主

建议：团队若多人、多供应商，方法一更稳更省心；若只用一个工具、想完全自主，方法二也足够。无论哪种，API Key 都属敏感信息，切勿提交进仓库。

十三、一句话收尾

“数据结构/JSON 接口” = schemas.py 里的 Pydantic 模型，第 0 步定死，所有 Agent 只吃 JSON、吐 JSON，由 coordinator.py 串成主链路；目录按“数据/逻辑/基础设施/接口”分层是为了让多人并行且互不踩。前期脚手架已给出“能跑的骨架”，但定契约细节、编写/迭代 Prompt、补齐 Agent 输出校验、每日真实跑通这几件最需由搭建者亲自动手的大事仍为空缺——尤其是 planner.py / matcher.py / timeline.py 中的三处 # TODO: 输出校验。

零、写在最后：两个常见疑问

初读本项目的人常有两个疑问，放在卷末统一回答，便于先建立整体认知再回看。

疑问一：不用框架（如

LangChain）是否就等于“手搓”，而用了框架就等于它把很多底层写好、我们只填业务？

- 基本是这个意思，但更精确地说：
- “不用框架” = 把 LLM 接入、输出解析、步骤编排、重试容错这些“脏活”自己用 openai + pydantic 写出来（本项目就是这么干的，requirements.txt 中没有 langchain）。
- “用框架” = LangChain 已把上述零件做成现成积木（ChatModel / PromptTemplate / OutputParser / Chain / Agent / Memory / Retriever），主要做拼装 + 填业务内容，不用自己写底层循环。
- 关键区别不在“能力”，而在“可控性 vs 速度”。手搓初期慢、但每一步都看得懂；用框架快、但出 bug 时得钻框架内部。对课程作业，手搓反而更能讲清“每一层是自主设计的”，答辩更显架构功力。

疑问二：整个 Agent 的搭建思想是否就是这套结构，剩下无非是写代码 + 写提示词，想再深就去看源码？

- 对，骨架思想就一句话，记住它便通了：先定死“数据结构 / JSON 契约”（schemas.py）→ 把“怎么调模型”封装成一个干净的类（llm/client.py）→ 每个 Agent 只干“吃一种 JSON、吐一种 JSON”（agents/*.py）→ 用一个 Coordinator 把多个 Agent 串成主链路（coordinator.py）→ 最后套一个 Web 壳把请求转成输入（main.py / web）。
- 剩下的工作确实就是三件：① 把代码逻辑写对（尤其是输出校验、容错降级）；② 把提示词（prompts.py）反复打磨到模型吐出的 JSON 稳定；③ 想再深，就顺着上面这条链路去读每个文件、跑一次真实调用、改一处看连锁反应。本手册前面各章就是把这条链路逐层拆开讲。

—— 本手册整合自项目历次讲解（含最初三张架构图），供本人及队友反复查阅。